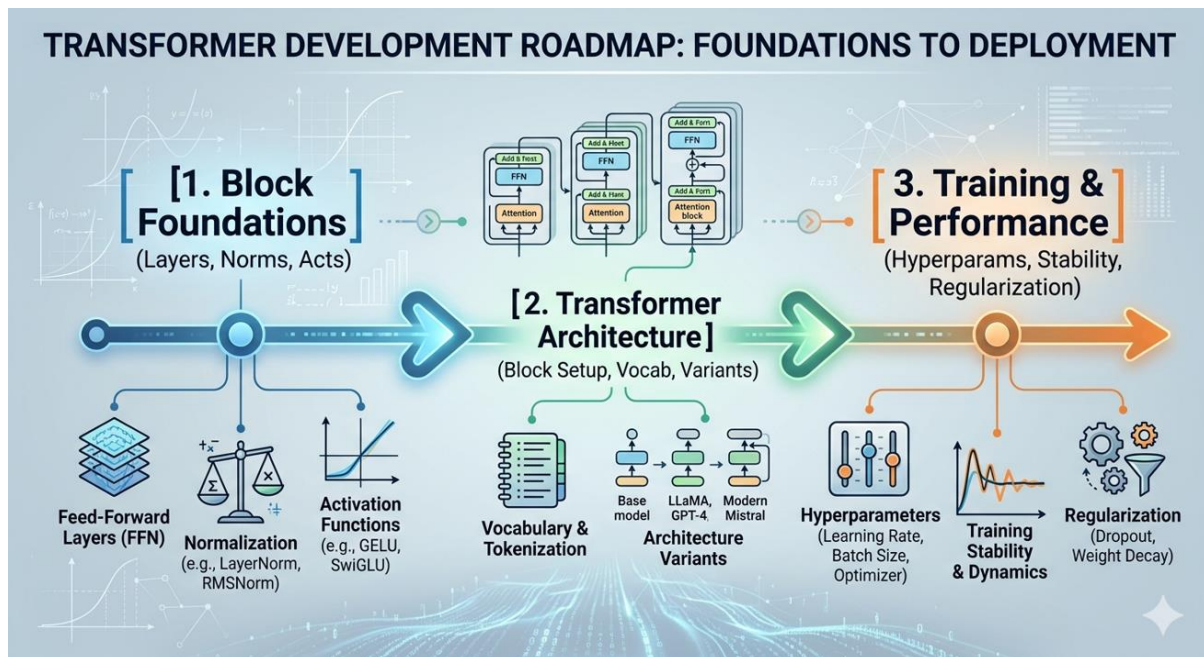


# Transformerni rivojlantirish xaritasi: poydevordan realni ishlatishgacha

## Arxitektura 2 qism



LL model yaratishdagi Arxitekturaning bu qismi 3 qismga bolinadi

1. Block Foundations
2. Transformer
3. Training & Performance

Bu yerdan Block Foundationsni ozi 3ga bolinadi:

1. Feed Forward Network Layerlar (FFN - Feed Forward Network)
2. Normalization
  - Layer Normalization
  - RMSNormalization

### 3.Activation Functions

Relu

Gelu

Silu

SwiGLU

### Transformer Architecture — Transformer arxitekturası

Bunga:

- Vocabulary & Tokenization;
- Block Setup;
- Architecture Variants

kiradi.

### Training & Performance — o‘qitish va samaradorlik

Bunga:

- Hyperparameters;
- Training Stability & Dynamics;
- Regularization

kiradi.

Layerlar nima?

Layerlar bu matematik matematik formulalar toplamidir. Yani input kiradi, hidden layerlarda matematik hisoblar bajariladi, output layerda

ham hidden layerlarda qayta qayta ishlanib hisob kitob qilingan input output layerda ham hisob kitob qilinib output bolib chiqadi. Input outputga weightlar yordamida otadi

LLM NN da layerlar 2ta qismdan iborat bular:

1.Attention layer (Context Engine)

tokenlar bir-biri bilan gaplashadi va orasidagi bog‘lanishni hisoblaydi gapdagi so‘zlarni bir-biriga bog‘lab umumiy ma’noni aniqlaydi

2.Feed Forward Network(FFN) layer ( Knowledge,bilim Engine)

har bir tokenni mustaqil qayta ishlab, uning ichki representationini yangilaydi

Ishlash ko‘lami

Attention Global:

matndagi barcha tokenlar orasida ishlaydi(Full Attentionda)

FFN Local:

bitta tokenning vector dimensionlari ichida ishlaydi.

GPU matematikasi Attentionda:

sequence/tokenlar bo‘ylab aralashtiradi.

FFN:

feature dimensionlar bo‘ylab aralashtiradi.

Nima o'rganadi?

Attention:

- grammatika;
- sintaksis;
- so'zlar orasidagi munosabat

kabi narsalarni organadi

FFN:

- dunyoqarash;
- aniq faktlar;
- mantiqiy fikrlashga oid bogliqliklar

kabi narsalarni organadi

Zamonaviy texnologiyalar qaysini ishlatadi hozirda?

Attentiondan:

GQA — Grouped-Query Attention.

FFNdan:

SwiGLU — Gated Linear Unit oilasidagi activation/gating mexanizmini

Faktlar faqat FFNda saqlanadi deb tushunish bu notogri.

Modelning bilimi embeddinglar, Attention weightlar, FFN weightlar va ko'plab layerlar

bo'ylab distributed, taqsimlangan tarzda saqlanadi.

Logit — vocabularydagi har bir token uchun model hosil qilgan birinchi xom scorelar

Biz Layerlarni joylashtirishni 2 xil usulidan birini tanlashimiz mumkin bular:

1.Serial( Attention layer 1chi ishlab tugagandan keyin FFN layer ishlaydi)

Hamma malumot Attention qatlamidan o'tadi keyin chiqish qiymati FFN qatlamiga kiradi

Kod yozish va tushunish oson

Katta modellarni oqitsa juda ko'p xarajat ketadi

Training vaqti ko'proq

2.Parallel( Attention layer ham FFN layer ham bir vaqtda ishlaydi)

Hamma malumot birdaniga ikkiga bo'linib Attention va FFN qatlamlariga parallel kiradi

Bitta inputdan Attention va FFN yo'llari ajralib ketadi.

Attention va FFN bir-birining outputini kutmaydi.

Parallel ishlagani uchun kod yozish murakkabroq

Sanoat standartidagi barcha LLM modellar shundan foydalanadi

Training vaqtini qisqartiradi

Normalization bu Scalingning boshqa bir nomidir.

Yani NN ichidagi sonlarni kichik sonlar oraligida ushlab turadi.nega kichik.chunki agar sonlar katta bolsa Training davomida sonlar,weightlar borgan sari kattalashib portlashi mumkin bu exploding gradients deyiladi yani sonlar borgan sari kattalashib portlaydi va xotira tolib ketadi va model ishdan chiqishi mumkin.

Hozirda Transformerlar Normalizationni asosan 2ta turini ishlatadi bular:

- 1.LayerNorm(eski usul)
- 2.RMSNorm(sanoat standarti)

LayerNorm

1. LayerNorm nima qiladi?

LayerNorm avval:

Bu sonlarning o'rtasi qayerda? deb qaraydi.

Bizda 10, 20, 30 sonlar bor deylik

O'rtachasi hisoblasak

$$(10+20+30)-3=20$$

Demak markaz 20

Endi LayerNorm har bir sondan 20ni ayiradi:

$$10-20=-10, 20-20=0, 30-20=10$$

Natijada

$$[-10, 0, 10]$$

Oldin:

$$[10, 20, 30] \text{ edi.}$$

Endi  $[-10, 0, 10]$  bo'ldi.

LayerNorm nima qildi bu yerda? Sonlarning markazini 0 ning atrofiga ko'chirdi. Keyin ularni yana mos scalega keltiradi.

Hozircha LayerNorm faqat **centering**, ya'ni markazlashtirish qildi.

Endi scaling boshlanadi.

**Variance topamiz**

Variance — qiymatlar o'rtachadan qanchalik tarqalganini o'lchaydi.

Shuning uchun LayerNormdagi **scale qilish**:  
har bir qiymatni standard deviation'ga bo'lish.

Variance:

$$\text{Variance} = [(-10)^2 + 0^2 + 10^2] / 3$$

$$\text{Variance} = (100 + 0 + 100) / 3$$

$$\text{Variance} = 200 / 3$$

$$\text{Variance} \approx 66.67$$

Standard deviation:

$$\text{Standard deviation} = \sqrt{66.67}$$

$$\text{Standard deviation} \approx 8.165$$

Endi har bir qiymatni 8.165 ga bo'lamiz:

$$-10 / 8.165 \approx -1.225$$

$$0 / 8.165 = 0$$

$$10 / 8.165 \approx 1.225$$

Shuning uchun LayerNorm natijasi:

$$[-1.225, 0, 1.225]$$

LayerNorm markazni 0 ga olib keladi va kattalikni tartiblaydi.

## **RMSNorm misoli**

Boshlang'ich qiymatlar:

$$\mathbf{x} = [10, 20, 30]$$

Har birini kvadrat qilamiz:

$$10^2 = 100$$

$$20^2 = 400$$

$$30^2 = 900$$

Kvadratlarning o'rtachasi:

$$(100 + 400 + 900) / 3$$

$$= 1400 / 3$$

$$\approx 466.67$$

Kvadrat ildiz olamiz:

$$\mathbf{RMS} = \sqrt{466.67}$$

$$\mathbf{RMS} \approx 21.60$$

Endi har bir qiymatni 21.60 ga bo'lamiz:

$$10 / 21.60 \approx 0.463$$

$$20 / 21.60 \approx 0.926$$

$$30 / 21.60 \approx 1.389$$

Shuning uchun RMSNorm natijasi:

$$[0.463, 0.926, 1.389]$$

| LayerNorm vs RMSNorm: Taqqoslash Jadvali |                                                       |                                                                 |
|------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------|
| Xususiyati                               | LayerNorm (Klassik)                                   | RMSNorm (Zamonaviy)                                             |
| Hisoblash usuli                          | O'rtacha qiymat (Mean) + Variatsiyani hisoblaydi.     | Faqat Kvadratik O'rtacha (RMS) qiymatni hisoblaydi.             |
| GPU Tezligi                              | Standart tezlikda ishlaydi (sekinroq).                | ~10% dan 50% gacha tezroq (matematik amallar kamligi evaziga).  |
| Model Aniqligi                           | Barqaror va yaxshi.                                   | LayerNorm bilan bir xil aniqlikda, lekin resursni juda tejaydi. |
| Qayerda ishlatiladi?                     | Eski GPT modellarida, BERT va original Transformerda. | Llama 3, GPT-4, Claude 3, Gemini va barcha yangi LLMlarda.      |

Activation Function nima?

Activation Function — neyron tarmoqdagi sonlarni qayta ishlaydigan va modelga non-linearity, ya'ni murakkab, chiziqli bo'lmagan bog'lanishlarni o'rganish imkonini beradigan funksiyadir

Yanada aniqrogi:

Linear layer ma'lumotni aralashtiradi, Activation Function esa unga murakkablik qo'shadi.

modelga qancha linear layer qo'yilsa ham murakkab non-linear patternlarni yaxshi o'rgana olmaydi. shuning uchun linear layerlar orasiga activation function qoyiladi

U neural networkka:

murakkab nonlinear funksiyalarni o'rganish imkonini beradi.

Yana qaysidir ma'noda activation function modelni reasoning(mantiqiy fikrlash) qobiliyatiga ham hissa qoshadi. yani foydalilarini otkazib foydasizlarini otkazmaydi(sozlarni)

Haqiqiy Transformerda activation odatda, FFNning ichida bo‘ladi

## Block Foundations : Activation Functions

• Uning turlari:

• ReLU

• GELU

• SwinGLU

• GEGLU

| Turi   | Qisqacha Formulasi              | LLMdagi Holati                                                       | Kimlar Ishlatadi?         |
|--------|---------------------------------|----------------------------------------------------------------------|---------------------------|
| ReLU   | $\max(0, x)$                    | Eskirgan. Neyronlarni tashlab yuborgani uchun LLMda ishlatilmaydi.   | Klassik chuqur tarmoqlar. |
| GELU   | $x \cdot \Phi(x)$               | Yaxshi. Ilk Transformerlarning asosi bo‘lgan silliq funksiya.        | GPT-3, BERT.              |
| SwiGLU | $\text{Swish}(xW) \otimes (xV)$ | Eng zo‘r! Sanoat standarti, eng tez va aqlli darvozali tizim.        | LLaMA 3, Gemini, Mistral. |
| GEGLU  | $\text{GELU}(xW) \otimes (xV)$  | Yaxshi. SwiGLU kabi kuchli darvozali variant, lekin bir oz sekinroq. | PaLM, T5.                 |

GEGLU va SwiGLUning asosiy ishi shundaki, ular faqat feature yaratmaydi — modelga har bir input uchun qaysi featurelarni ko‘proq yoki kamroq o‘tkazishni ham o‘rgatadi.

## Vocabulary va Tokenization

Tokenization – bu matnni tokenizer vocabularysidagi token birliklariga ajratishdir

Vocabulary – bu Tokenizer taniydigan barcha tokenlar va ularning IDlari ro‘yxatidir.

Tokenization — bu algoritm

Gapni o'qiydi va bo'laklarga ajratadi.

Vocabulary — bu lug'at ombori

har bo'lakni o'z ro'yxatidan qidiradi

| Bosqich  | Qaysi biri ishlaydi?       | Nima sodir bo'ladi?                                                                                                                                   |
|----------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1- Qadam | Tokenization (Algoritm)    | Gapni o'qiydi va uni lug'atga moslab maydalaydi: "o'z", "bek", "iston" bo'laklariga ajratadi.                                                         |
| 2- Qadam | Vocabulary (Lug'at ombori) | Ajratilgan bo'laklarni o'zining ichki ro'yxatidan qidiradi va ularning ID raqamlarini qaytaradi: "o'z" → ID: 412, "bek" → ID: 3901, "iston" → ID: 55. |

## Pre-Norm va Post-Norm

Pre-Norm va Post-Norm bu Transformer blockda Normalization qayerda turishini ikki usulga ajratishdir

Pre = oldin.

Post = keyin.

Pre-Norm

Input ↓ Tokenization ↓ Normalization ↓ Attention / FFN ↓ Activation

Asosiy tushuncha bu yerda:

Normalization sublayer(Attention / FFN)dan oldin.

## Post-Norm

Asosiy tushuncha bu yerda:

Input ↓ Tokenization ↓ Attention / FFN ↓ Normalization ↓ Activation

Attention/FFN avval, keyin Normalization

## Pre-Norm nega mashhur?

Juda chuqur Transformer trainingda residual path orqali gradientning yurishi osonroq bo‘lishi mumkin.

Shuning uchun ko‘p katta LLM architecturelari pre-normalization oilasidan foydalanadi.